Can a small local agent redo a cloud agent's work?

93 evaluator-verified tasks from an agent-built health-data repository, with an open harness and a Gemma 4 E4B baseline

John Bowyer, Health Transformation Lab

Gemma 4 Developer Agent Competition, Paper Track [14].
Topic: Tasks & Benchmarks (resource paper). November 2026.

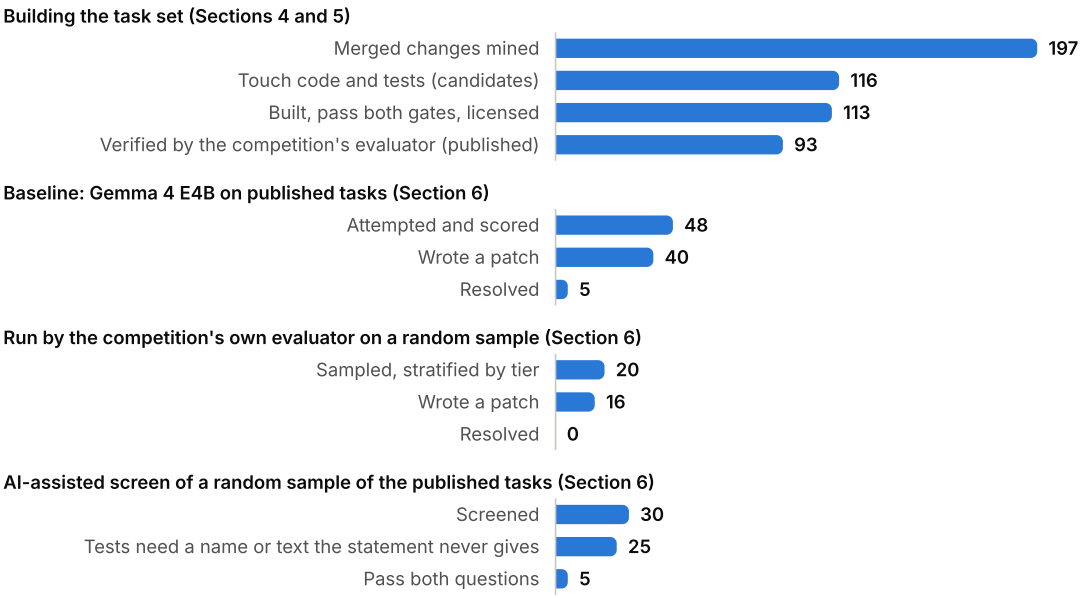
A personal learning project, not affiliated with or endorsed by any employer, institution or vendor. Every patient record in the source repository is synthetic.

ABSTRACT

In fifteen days, one person in the loop with a cloud coding agent took a health-data repository from nothing to 960 commits (775 marked as agent-authored) and 2,269 tests, without typing code. Scripts turn its CI-enforced record into 93 health-data modelling tasks made the way the competition's own are: its mining rule, its format, its evaluator on every task. What can a small local model take out of that loop? **What it did unattended:** on free T4 GPUs, Gemma 4 E4B resolved 5 of the 48 tasks it ran, cheapest first (95% CI 4.5–22.2%), all small. With eight times the output budget it resolved 2 of the 23 cheapest, against 2 before. Under the competition's evaluator it resolved none of a random 20. **What the graph could tell it:** by name, call edge or embedding search (first ten results), the graph leads from statement to fix in 42 of the 51 tasks that edit existing code, and embedding search ranks an edited function first in 15 (0.6 by chance). The model asked a graph tool in only 20 of 473 calls. **Where its time paid:** only 17 of 93 statements say what their tests check (an AI screen, matching all 30 a person checked); the 13 it ran gave 4 of its 5 fixes for 17% of its time, and the tasks a name or call edge reaches gave 3 for 20%.

Data and code. Tasks (CC BY 4.0): kaggle.com/datasets/jcblearning/health-transformation-lab-coding-agent-tasks.
Quickstart notebook: kaggle.com/code/jcblearning/coding-agent-tasks-quickstart.
Harness, builders and run tables (Apache 2.0): github.com/healthtransformationlab/health-transformation-lab-nlp.
Agent configuration and each number's source (Apache 2.0): github.com/healthtransformationlab/gemma-developer-agent.

Figure 1. Task counts at each step, on one scale.



1. Introduction

This competition [14] asks whether a model that fits on one accelerator can do the work of the large cloud models behind today's best coding agents. Available tasks come from a narrow place: SWE-bench draws on twelve popular Python libraries [1], and the competition's development set on four.

Two kinds of repository are scarce. In a **data repository** the costly defect is silent: a pipeline finishes and a number in its table is wrong. An **agent-built repository** can leave an unusually complete record: an issue, a pull request and a review for each change. Which changes can a small local agent take out of the person's loop, and what does the person need for the rest?

This paper adds one repository that is both, set beside the competition's four by the competition's mining rule, format and evaluator. `health-transformation-lab-nlp` exists to surface silent defects, here in how well gum disease can be found in synthetic dental records, so its tasks are real changes to application code: de-identification, record checks, model scoring, pipelines. Every change was made by coding-agent sessions directed in chat and merged after an automated review, and every step from merged change to published task is a script.

Contributions:

1. **A task set**: 93 tasks from health-data modelling work in the competition's format, so its harness, evaluator and graph tools run on both sets; each verified with the competition's own evaluator and labelled with its statement's source (Section 4).
2. **An open harness** that agrees with the competition's evaluator on every saved agent patch and keeps answers from the model (Section 5).
3. **A baseline under three settings** for Gemma 4 E4B, with a failure breakdown, a paired test of the code-graph tools, and its time split by graph reach (Section 6).
4. **Three checks on the recipe**: whether statements say what their tests check (Section 6), whether names, call edges or embedding search lead from statement to fix (Section 4.3), and a second repository (Section 4.5).

2. Related work

Task sets from repository history. SWE-bench pairs GitHub issues with the pull requests that closed them and scores a patch by the repository's tests [1]; SWE-bench Verified adds human screening of task quality [2]. SWE-Gym [5], SWE-smith [6] and SWE-rebench [7] scale task collection for training and for contamination-free evaluation. These draw on public open-source projects; this set comes from an agent-built data repository and adds statement-source labels and counts of how often the graph leads to the fix.

Agents and graphs. SWE-agent showed that the interface matters as much as the model [8]. RepoGraph [3] and LocAgent [4] lead an agent to the code to edit by finding an issue's terms in a code graph and walking its edges; Section 4.3 counts how often such a walk exists here.

Data work. DS-1000 tests data-science code generation [9]; Spider 2.0 [10], ELT-Bench [11] and data-eng-bench [13] score an artefact agents produce: SQL, a pipeline, dbt models. MLOps-Bench [12] is closest: 212 fail-to-pass tasks from 31 public ML repositories.

3. The source repository

The repository holds a Python package and tests, an R twin, SQL and Databricks pipelines, and documentation, written October 25 to November 8, 2026.

Who wrote it. Of 960 commits on `main`, 775 are marked as agent-authored; the rest are GitHub's merge commits for pull requests (175), a bot's (5), the author's alone (4) and the initial commit.

Rules the agents work under. A checked-in `AGENTS.md` requires an issue per user story, a pull request tied to it and a test before the code; CI rejects a pull request without an issue link. Each pull request has a committed review (240 of them). On November 8 it held 2,269 tests in 126 files, run by CI on every pull request.

4. Building the task set

4.1 Mining

The mining script reads every merged pull request and every first-parent commit on `main` as of October 29, 2026: 197 changes. A change is a **candidate** when its diff touches both non-test code and test files, which is the competition's own mining rule.

Step	Tasks
Merged changes	197
Touch code and tests (candidates)	116
Built (one withdrawn under the repository's rules)	115
Pass both gates	114
One withheld (unlicensed third-party files)	113
Published: the competition's evaluator resolves the reference patch (Section 5)	93

4.2 Fidelity tiers

Each task records where its `problem_statement` came from.

Tier	Statement source	Candidates (published)
A	the issue body	49 (39)
B	the pull request's title and body (no issue existed)	45 (34)
C	the commit message (no pull request existed)	22 (20)

Tiers B and C exist because the issue rule was adopted partway through; for those 67 candidates a **backfilled issue record** quotes the original text.

4.3 Snapshots, graphs, embeddings

A snapshot is the tree at `base_commit` as a single commit, so no later history comes along. The code graph is an AST walk over the package into NetworkX node-link JSON with the competition's keys: a node for each function, class and method with its source text, and a `calls` edge where one calls another (medians: 488 nodes, 397 edges). Embeddings come from EmbeddingGemma, cut to 256 dimensions and renormalised. A format check against the competition's own files matched (layout, graph keys, embedding shape); the competition does not name its embedding model.

The graph reaches only part of what a data repository changes. It misses the fix in 42 of the 93 tasks: 31 touch no module it covers (new modules, scripts, notebooks, configuration or documents) and 11 change no existing function or class. In the other 51, the statement names an edited function or class (by its identifier) in 20, call edges lead there from one it names in 2 more, and in 29 neither does. Embedding search does more: ranking a graph's

nodes (median 398) by similarity to the statement puts an edited one first in 15 of the 51 (0.6 expected by chance: each graph's nodes ranked at random, hypergeometric, summed over tasks) and in the first ten in 30 (5.4 by chance), 20 of them among those 29. So by name, call edge or embedding search the graph leads to the fix in 42 of the 51. These are counts from the published files, not agent runs.

4.4 Gates

A task is kept only if its tests fail at `base_commit` with the test patch alone, and pass once the reference patch is applied with the files the grader strips removed. All 115 fail without the fix; 114 pass with it.

4.5 A second repository

Applied unchanged to Open Navigator, a public repository run by a nonprofit the author leads, with several human contributors and no issue-tracing rule, the mining rule finds 133 candidates in 1,320 first-parent commits. None links an issue and only 22 can be tied to a pull request: statements depend on what the history recorded. Gating and releasing them is future work.

5. Harness and verification

The local harness. This work re-implemented the harness the competition's guide documents before finding its evaluator in a separate Kaggle dataset. Two scripted models bound it: replaying each reference patch from the repository's history resolves 111 of 113 tasks (the other two need Postgres and R), and submitting nothing resolves none.

The competition's evaluator. The competition's own verification (`swegemma` 0.2.11, subprocess sandbox, unmodified) was then run on the 113 tasks first published and on the corrected release.

Snapshots	Reference patch	Empty patch	Gemma's saved patches
First release, with the import fix below	93 of 113 resolved	0 of 113	both verifiers agree on 25 of 25
Corrected release, as published	93 of 93 resolved	0 of 93	both verifiers agree on 23 of 23

As first published, no task could be scored (all three tried fail at import): the evaluator puts the repository root on the import path, and this package sits in `python/`. The corrected release adds a short `conftest.py` at each snapshot's root and keeps only the 93.

Of the 20 left out, twelve have tests that read third-party files the published snapshots withhold for licence reasons, six start a script that imports a module beside it, which the evaluator's `PYTHONSAFEPATH=1` forbids and the local harness allows, and two need Postgres or R.

Keeping answers from the model. This repository's history contains every fix, and later snapshots contain earlier tasks' fixes, so commands and verification run in Linux namespaces with no network and an allow-listed environment. On Kaggle, where namespaces are not allowed, each task runs alone in a private, offline notebook beside a private dataset holding that task only.

Known differences. Namespaces instead of a container; no ADK context compaction or prefix caching; subagents and skills are not run; `PYTHONSAFEPATH` is not set. No number here is a competition score.

6. Baseline

Setup. `gemma-4-e4b-it` in fp16 with transformers on two T4 GPUs, internet off, thinking off, at most 2,048 new tokens per turn, a 24,576-token context, 40 tool calls, 60 turns and 30 minutes per task. Tasks were run cheapest first under a GPU-hour cap, so no rate here is an unbiased estimate for the 93.

Results. Ten of the 58 tasks attempted were excluded from the set (Section 5). On the other 48 the model wrote a patch for 40 and resolved 5 (10.4%, 95% Wilson CI 4.5–22.2). By statement source: 4 of 24 from issues, 1 of 19 from pull requests, 0 of 5 from commit messages; the intervals overlap.

Do the graph tools help? A harness fault served the first tasks without their code graph; after the fix, 28 were re-run with the same settings. On the 23 published, 2 were resolved either way (one in common). The model made only 6 graph-tool calls among 145, in 4 of the 23 runs.

How it fails. Of the 43 unresolved tasks, 35 submitted a patch that fails the target tests and 8 submitted nothing. The five resolved tasks are small: a CI rule, a warning message, a label parser and two settings.

A larger budget. To test the tight limits, the 23 cheapest of the 48 were rerun on the same harness on one Colab L4 GPU: 16,384 output tokens per turn, thinking on, 100 tool calls, 20 minutes and a 20,000-token context. The model resolved 2 of 23, against 2 under the tight limits, and wrote a patch for 20 against 19. Of its 139 tool calls, 5 were to a graph tool. Colab had network access; no command used it.

Task fairness. Two AI screens rated tasks on SWE-bench Verified's two questions [2]. The first took a random 30; the author checked its findings in summary and agreed with all 30: five pass, and in 25 the tests need a name or text the statement never gives. An independent second pass, blind to the first, rated all 93: the same verdict on all 30, and 17 of 93 pass. Each verdict and its reason are in the repository (`data/tasks/screening/`). SWE-bench's own human screen filtered out 68.3% of 1,699 samples [2]. Of the 48 tasks the model attempted, it resolved 4 of the 13 that pass and 1 of the 35 that fail, so the baseline may measure the tasks more than the model.

Under the competition's evaluator. The agent was also run end to end with the official harness: `swegemma`'s agent loop, vLLM from the competition's wheelhouse, four L4 GPUs, and its competition settings (16,384 output tokens, a 4,096-token thinking budget, 40 tool calls, 5 minutes). The sample is 20 of the 93, seeded and stratified by tier, not by cost. Each task ran alone, without patch or tests. The agent resolved 0 of 20 (95% CI 0–16.1): 16 patches fail and 4 runs submit nothing. In 189 tool calls the model called a graph tool 9 times. Seventeen of the 20 fail the second screen.

Cost, and what to send. Of the agent's 9.0 hours on the 48 tasks, 17% went to the 13 that pass the second screen, which gave 4 of the 5 fixes, and 20% to the 12 a name or call edge reaches, which gave 3: 0.38 and 0.61 hours a fix, against 1.80 overall. The screen reads the tests, which a test-first workflow has before the fix; graph reach here uses the reference fix, so it is not yet a routing rule.

7. Limitations

- **One new repository.** It sits beside the competition's four under the same rule, format and evaluator; the screens and graph counts were run on it alone.
- **Agent-written statements, no cloud baseline.** A cloud agent wrote most of the issue text at the author's direction and made these changes with a person in the loop; it was not rerun on the statements alone.
- **No leaderboard score.** The competition's evaluator ran the agent on 20 tasks only, with non-default settings (5 minutes and 40 tool calls, not 60 and 100).

- **Small.** 48 of 93 tasks, cheapest first, and a random 20 for the evaluator; one run each at temperature 0.2.
- **AI screens.** A person checked 30 verdicts, in summary, after seeing the AI's; 63 have AI ratings only.
- **Prompted repair.** Each task comes with a failing test; the set does not measure noticing a silent defect unprompted.

8. Conclusion and release

From a cloud agent's own problem statements, a model that runs on free GPUs resolved 5 of 48 cheapest-first tasks, 2 of 23 with a larger budget and none of a random 20 under the competition's evaluator. The graph could have told it where to look in 42 of the 51 tasks that edit existing code, yet only 20 of 473 tool calls asked a graph tool: that signal is there to be used, its benefit to an agent untested. Sending the model only the tasks that pass the screen would have kept 4 of its 5 fixes for 17% of its time.

Using the set. The dataset has the competition's layout, so its evaluator runs on it as published. In the repository, `kaggle/official_verify.py` replays the two controls and scripts 63 to 66 the graph, search, cost and routing counts; the quickstart notebook loads a task with its snapshot, graph and embeddings.

The recipe is the competition's (changes that touch code and tests, its format, its evaluator) plus what this paper adds on one repository: record each statement's source, run reference-patch and empty-patch controls on the published files, screen statements against tests, and count where the graph leads to the fix.

References

1. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.
2. Chowdhury et al. Introducing SWE-bench Verified. OpenAI, 2024.
3. Ouyang et al. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. ICLR 2025.
4. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL 2025.
5. Pan et al. Training Software Engineering Agents and Verifiers with SWE-Gym. ICML 2025.
6. Yang et al. SWE-smith: Scaling Data for Software Engineering Agents. NeurIPS 2025.
7. Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. NeurIPS 2025.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Lai et al. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. ICML 2023.
10. Lei et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. ICLR 2025.
11. Jin, Zhu, Kang. ELT-Bench: An End-to-End Benchmark for Evaluating AI Agents on ELT Pipelines. PVLDB, 2026.
12. Tran et al. MLOps-Bench: Benchmarking AI Agents on Production ML Engineering in Real Repositories. arXiv:2610.23506, 2026.
13. Snowflake Inc. data-eng-bench: a data-engineering benchmark for coding agents. github.com/Snowflake-Labs/data-eng-bench, 2026.
14. Markowitz, Perozzi, Rózemberczki, Cameron, Hemmati, Li, Galkin, Farhadi, Holbrook, Oldacre. Google - The Gemma 4 Developer Agent Competition. Kaggle, 2026. https://www.kaggle.com/competitions/gemma-4-developer-agent